

Дополнение к ТЗ: Авторизация, регистрация и система ролей

1. Общая логика авторизации

В системе должна быть реализована авторизация пользователей через логин и пароль.

Backend:

- FastAPI
- JWT access token
- Refresh token
- Password hashing через bcrypt
- Role-based access control
- Проверка прав доступа на уровне API

Frontend:

- страница входа;
- страница регистрации или создания пользователя;
- защита приватных страниц;
- скрывание пунктов меню в зависимости от роли;
- redirect на login, если пользователь не авторизован.

2. Регистрация пользователей

2.1. Вариант для MVP

Для первой версии не делать открытую регистрацию для всех.

Правильная логика:

- обычный пользователь не может сам зарегистрироваться и попасть в систему;
- пользователя создает Super Admin или Boss;
- при создании пользователя ему назначается роль;
- пользователь получает логин/пароль;
- после первого входа может сменить пароль.

Это безопаснее, потому что CRM внутренняя.

2.2. Кто может создавать пользователей

Создавать новых пользователей могут только:

- Super Admin;
- Boss.

Остальные роли не могут создавать пользователей.

2.3. Поля пользователя

```
id UUID PK
full_name VARCHAR(255)
phone VARCHAR(50)
email VARCHAR(255) UNIQUE
password_hash TEXT
role VARCHAR(50)
is_active BOOLEAN DEFAULT true
must_change_password BOOLEAN DEFAULT true
last_login_at TIMESTAMP NULL
created_by UUID FK -> users.id NULL
created_at TIMESTAMP
updated_at TIMESTAMP
deleted_at TIMESTAMP NULL
```

3. Роли системы

В системе должно быть 5 ролей:

```
SUPER_ADMIN
BOSS
WAREHOUSE_MANAGER
SHIFT_MASTER
SALES_MANAGER
```

4. Описание ролей

4.1. SUPER_ADMIN

Технический владелец системы.

Имеет полный доступ ко всем разделам:

- пользователи;
- роли;
- клиенты;
- заказы;
- производство;
- отчеты смен;
- склад;
- закупки;
- расходы;
- оплаты;
- задолженности;
- P&L;
- dashboard;
- настройки;
- audit logs.

Может:

- создавать пользователей;
- менять роли;
- отключать пользователей;
- удалять записи;
- исправлять критичные ошибки;
- видеть все деньги и аналитику.

4.2. BOSS / Руководитель

Владелец или директор бизнеса.

Имеет полный бизнес-доступ:

- dashboard;
- клиенты;
- заказы;
- производство;
- отчеты смен;
- склад;
- отгрузки;
- оплаты;
- задолженности клиентов;
- расходы;
- P&L;
- отчеты;
- справочники.

Может:

- создавать пользователей;

- назначать роли, кроме SUPER_ADMIN;
- утверждать отчеты смен;
- утверждать расходы;
- смотреть прибыль;
- смотреть долги клиентов;
- смотреть все движения склада;
- смотреть все оплаты;
- смотреть все расходы.

Ограничение:

- не может удалить или изменить Super Admin;
 - не может назначить роль SUPER_ADMIN, если это не разрешено отдельно.
-

4.3. WAREHOUSE_MANAGER / Зав. склад

Сотрудник, отвечающий за склад.

Должен видеть:

- остатки готовой продукции;
- остатки сырья;
- движения склада;
- приход сырья;
- расход сырья;
- отгрузки товара;
- ручные корректировки склада, если разрешено;
- список товаров и материалов.

Не должен видеть:

- P&L;
- чистую прибыль;
- общие расходы;
- зарплаты;
- закупочные суммы, если это скрыто настройками;
- оплаты клиентов;
- долги клиентов;
- финансовый dashboard.

Разрешенные действия:

- смотреть склад;
- подтверждать фактическую отгрузку;
- видеть, хватает ли товара на складе;
- делать складскую корректировку, если есть permission;
- принимать сырье на склад, если операция уже утверждена Boss;
- смотреть историю движений склада без финансовой аналитики.

Запрещенные действия:

- создавать расходы;
 - утверждать расходы;
 - смотреть общую прибыль;
 - смотреть оплаты клиентов;
 - смотреть долги клиентов;
 - менять цены заказов;
 - создавать пользователей.
-

4.4. SHIFT_MASTER / Мастер смены

Сотрудник, который заполняет отчет о производстве за смену.

Должен видеть только:

- свои отчеты смен;
- черновики своих отчетов;
- список товаров для выбора в форме;
- список материалов для выбора в форме;
- список работников смены, если нужно;
- статус своего отчета: черновик, отправлен, утвержден, отклонен.

Не должен видеть:

- общий склад;
- точные остатки склада;
- деньги;
- расходы;
- оплаты;
- долги клиентов;
- P&L;
- клиентов;
- закупки;
- прибыль;
- чужие сменные отчеты, если это не разрешено.

Разрешенные действия:

- создать отчет смены;
- выбрать смену 1 или смену 2;
- указать работников;
- указать произведенную продукцию;
- указать потраченное сырье;
- указать брак;
- сохранить черновик;
- отправить отчет на проверку;
- исправить отчет, если он отклонен.

Запрещенные действия:

- утверждать свой отчет;
- видеть финансовые данные;
- видеть складские остатки;
- создавать заказы;
- создавать расходы;
- создавать оплаты;
- смотреть долги клиентов;
- создавать пользователей.

Важный момент:

Мастер смены может выбирать сырье в форме, но не должен видеть полный складской остаток. Максимум можно показать предупреждение:

Материал выбран

Но не показывать:

На складе осталось 800 кг

Если бизнес хочет, можно разрешить показывать только статус:

Достаточно / Мало / Нет в наличии

Но без точных цифр.

4.5. SALES_MANAGER / Менеджер продаж

Сотрудник, который ведет клиентов, заказы, отгрузки и оплаты.

Должен видеть:

- своих клиентов;
- заказы клиентов;
- статусы заказов;
- готовность к отгрузке;
- отгрузки по заказам;
- оплаты клиентов;
- задолженность клиентов;
- остатки готовой продукции в ограниченном виде.

Может:

- создавать клиентов;

- создавать заказы;
- менять статус заказа;
- создавать отгрузку;
- добавлять оплату;
- смотреть долг клиента;
- смотреть историю клиента;
- видеть, есть ли товар на складе для отгрузки.

Не должен видеть:

- расходы компании;
- зарплаты;
- P&L;
- чистую прибыль;
- закупочные цены;
- себестоимость;
- полный склад сырья;
- производственные внутренние списания сырья;
- отчеты всех смен, если это не нужно.

Остатки для менеджера:

Менеджер может видеть готовую продукцию только в формате:

Есть в наличии / Мало / Нет в наличии

или, если бизнес разрешает:

Точный остаток готовой продукции

Но менеджер не должен видеть сырье, закупочные цены и себестоимость.

5. Таблица доступов по модулям

Модуль	Super Admin	Boss	Зав. склад	Мастер смены	Менеджер
Dashboard общий	Да	Да	Нет	Нет	Ограниченно
Пользователи	Да	Да	Нет	Нет	Нет
Клиенты	Да	Да	Нет	Нет	Да
Заказы	Да	Да	Только отгрузка	Нет	Да

Модуль	Super Admin	Boss	Зав. склад	Мастер смены	Менеджер
Производство	Да	Да	Ограниченно	Только свои отчеты	Ограниченно
Отчеты смен	Да	Да	Нет	Только свои	Нет
Утверждение смен	Да	Да	Нет	Нет	Нет
Склад готовой продукции	Да	Да	Да	Нет	Ограниченно
Склад сырья	Да	Да	Да	Нет	Нет
Движения склада	Да	Да	Да	Нет	Нет
Отгрузки	Да	Да	Да	Нет	Да
Оплаты клиентов	Да	Да	Нет	Нет	Да
Долги клиентов	Да	Да	Нет	Нет	Да
Расходы	Да	Да	Нет	Нет	Нет
Закуп сырья	Да	Да	Ограниченно	Нет	Нет
P&L	Да	Да	Нет	Нет	Нет
Настройки	Да	Ограниченно	Нет	Нет	Нет
Audit logs	Да	Да	Нет	Нет	Нет

6. Навигация по ролям

6.1. Меню для Super Admin

- Dashboard
- Клиенты
- Заказы
- Производство
- Отчеты смен
- Склад
- Отгрузки
- Оплаты
- Долги
- Расходы
- Отчеты P&L
- Пользователи
- Настройки

- Audit Logs
-

6.2. Меню для Boss

- Dashboard
 - Клиенты
 - Заказы
 - Производство
 - Отчеты смен
 - Склад
 - Отгрузки
 - Оплаты
 - Долги
 - Расходы
 - Отчеты P&L
 - Пользователи
 - Настройки
-

6.3. Меню для Зав. склад

- Склад
- Остатки
- Движения склада
- Отгрузки
- Приемка
- Корректировки

Не показывать:

- Dashboard с деньгами;
 - Расходы;
 - Оплаты;
 - Долги;
 - P&L;
 - Пользователи.
-

6.4. Меню для Мастера смены

- Моя смена
- Мои отчеты
- Новый отчет смены
- Профиль

Не показывать:

- Склад;
- Деньги;

- Заказы;
 - Клиенты;
 - Расходы;
 - Отчеты;
 - Пользователи.
-

6.5. Меню для Менеджера продаж

- Мой Dashboard
- Клиенты
- Заказы
- Отгрузки
- Оплаты
- Долги клиентов
- Готовая продукция
- Профиль

Не показывать:

- Расходы;
 - P&L;
 - Склад сырья;
 - Себестоимость;
 - Зарплаты;
 - Пользователи.
-

7. Backend permissions

Нужно реализовать проверку прав на backend, а не только скрывать кнопки на frontend.

Frontend скрывает меню для удобства, но безопасность должна быть на backend.

Пример dependency в FastAPI:

```
def require_roles(*allowed_roles: UserRole):
    async def checker(current_user: User = Depends(get_current_user)):
        if current_user.role not in allowed_roles:
            raise HTTPException(
                status_code=403,
                detail="Недостаточно прав"
            )
        return current_user
    return checker
```

Пример использования:

```
@router.get("/expenses")
async def get_expenses(
    current_user: User = Depends(require_roles(
        UserRole.SUPER_ADMIN,
        UserRole.BOSS
    ))
):
    ...
```

8. Enum ролей в backend

```
from enum import Enum

class UserRole(str, Enum):
    SUPER_ADMIN = "super_admin"
    BOSS = "boss"
    WAREHOUSE_MANAGER = "warehouse_manager"
    SHIFT_MASTER = "shift_master"
    SALES_MANAGER = "sales_manager"
```

9. Матрица API-доступов

Auth

```
POST /api/auth/login – все
GET /api/auth/me – все авторизованные
POST /api/auth/logout – все авторизованные
POST /api/auth/change-password – все авторизованные
```

Users

```
GET /api/users – SUPER_ADMIN, BOSS
POST /api/users – SUPER_ADMIN, BOSS
PATCH /api/users/{id} – SUPER_ADMIN, BOSS
DELETE /api/users/{id} – SUPER_ADMIN
PATCH /api/users/{id}/role – SUPER_ADMIN, BOSS с ограничением
```

Boss не может:

- назначить SUPER_ADMIN;
- редактировать SUPER_ADMIN;
- удалить SUPER_ADMIN.

Clients

```
GET /api/clients – SUPER_ADMIN, BOSS, SALES_MANAGER
POST /api/clients – SUPER_ADMIN, BOSS, SALES_MANAGER
GET /api/clients/{id} – SUPER_ADMIN, BOSS, SALES_MANAGER
PATCH /api/clients/{id} – SUPER_ADMIN, BOSS, SALES_MANAGER
DELETE /api/clients/{id} – SUPER_ADMIN, BOSS
```

Ограничение:

Менеджер может видеть только своих клиентов, если включен режим `manager_scope`.

Orders

```
GET /api/orders – SUPER_ADMIN, BOSS, SALES_MANAGER
POST /api/orders – SUPER_ADMIN, BOSS, SALES_MANAGER
GET /api/orders/{id} – SUPER_ADMIN, BOSS, SALES_MANAGER
PATCH /api/orders/{id} – SUPER_ADMIN, BOSS, SALES_MANAGER
PATCH /api/orders/{id}/status – SUPER_ADMIN, BOSS, SALES_MANAGER
DELETE /api/orders/{id} – SUPER_ADMIN, BOSS
```

Shift Reports

```
GET /api/shift-reports – SUPER_ADMIN, BOSS
GET /api/shift-reports/my – SHIFT_MASTER
POST /api/shift-reports – SHIFT_MASTER, SUPER_ADMIN, BOSS
GET /api/shift-reports/{id} – SUPER_ADMIN, BOSS, owner SHIFT_MASTER
PATCH /api/shift-reports/{id} – owner SHIFT_MASTER только если DRAFT или REJECTED
POST /api/shift-reports/{id}/submit – owner SHIFT_MASTER
POST /api/shift-reports/{id}/approve – SUPER_ADMIN, BOSS
POST /api/shift-reports/{id}/reject – SUPER_ADMIN, BOSS
DELETE /api/shift-reports/{id} – SUPER_ADMIN, BOSS
```

Stock

```
GET /api/stock/balances – SUPER_ADMIN, BOSS, WAREHOUSE_MANAGER
GET /api/stock/materials – SUPER_ADMIN, BOSS, WAREHOUSE_MANAGER
GET /api/stock/products – SUPER_ADMIN, BOSS, WAREHOUSE_MANAGER, SALES_MANAGER
limited
GET /api/stock/movements – SUPER_ADMIN, BOSS, WAREHOUSE_MANAGER
POST /api/stock/adjustments – SUPER_ADMIN, BOSS, WAREHOUSE_MANAGER if allowed
```

Ограничение для SALES_MANAGER:

Менеджеру показывать только готовую продукцию. Не показывать сырье, закупочные цены, себестоимость и внутренние движения склада.

Shipments

```
GET /api/shipments – SUPER_ADMIN, BOSS, WAREHOUSE_MANAGER, SALES_MANAGER
POST /api/shipments – SUPER_ADMIN, BOSS, SALES_MANAGER
POST /api/shipments/{id}/confirm – SUPER_ADMIN, BOSS, WAREHOUSE_MANAGER
POST /api/shipments/{id}/cancel – SUPER_ADMIN, BOSS
```

Логика:

- менеджер создает отгрузку;
 - зав. склад подтверждает фактическую выдачу;
 - после подтверждения склад уменьшается.
-

Payments

```
GET /api/payments – SUPER_ADMIN, BOSS, SALES_MANAGER
POST /api/payments – SUPER_ADMIN, BOSS, SALES_MANAGER
PATCH /api/payments/{id} – SUPER_ADMIN, BOSS
DELETE /api/payments/{id} – SUPER_ADMIN, BOSS
```

Зав. склад и мастер смены не имеют доступа к оплатам.

Debts

```
GET /api/reports/debts – SUPER_ADMIN, BOSS, SALES_MANAGER
GET /api/clients/{id}/balance – SUPER_ADMIN, BOSS, SALES_MANAGER
```

Ограничение:

Менеджер может видеть долги только своих клиентов, если включен `manager_scope`.

Expenses

```
GET /api/expenses – SUPER_ADMIN, BOSS
POST /api/expenses – SUPER_ADMIN, BOSS
PATCH /api/expenses/{id} – SUPER_ADMIN, BOSS
POST /api/expenses/{id}/approve – SUPER_ADMIN, BOSS
POST /api/expenses/{id}/reject – SUPER_ADMIN, BOSS
DELETE /api/expenses/{id} – SUPER_ADMIN
```

Зав. склад, мастер смены и менеджер не имеют доступа к расходам.

Reports / P&L

```
GET /api/reports/dashboard – SUPER_ADMIN, BOSS
GET /api/reports/pnl – SUPER_ADMIN, BOSS
GET /api/reports/revenue-expense-trend – SUPER_ADMIN, BOSS
GET /api/reports/expenses-by-category – SUPER_ADMIN, BOSS
```

Остальные роли не имеют доступа.

10. Frontend Guard

На frontend нужно сделать защиту маршрутов.

Пример логики:

```
Если пользователь не авторизован:
    redirect /login

Если роль не имеет доступа к странице:
    показать 403 или redirect на свою главную страницу
```

Главная страница после входа зависит от роли:

```
SUPER_ADMIN -> /dashboard
BOSS -> /dashboard
WAREHOUSE_MANAGER -> /stock
```

```
SHIFT_MASTER -> /shift-reports/my  
SALES_MANAGER -> /orders
```

11. Login Page

Страница входа должна содержать:

- логин/email;
- пароль;
- кнопка "Войти";
- показать ошибку при неправильном пароле;
- loading state;
- адаптация под телефон.

После успешного входа:

- сохранить access token;
- получить `/api/auth/me`;
- сохранить пользователя;
- перенаправить на страницу по роли.

12. User Management Page

Доступно только:

- Super Admin;
- Boss.

Функции:

- список пользователей;
- создать пользователя;
- изменить роль;
- отключить пользователя;
- сбросить пароль;
- фильтр по роли;
- поиск по имени/телефону/email.

Поля создания пользователя:

ФИО
Телефон
Email
Роль

Временный пароль
Активен / не активен

13. Важные правила безопасности

1. Нельзя доверять frontend.
2. Все права проверять на backend.
3. JWT должен содержать user_id, но роль лучше проверять из базы.
4. Пароли хранить только в виде hash.
5. Super Admin нельзя удалить обычным способом.
6. Boss не может назначать Super Admin.
7. Мастер смены не может видеть чужие отчеты, если это не разрешено.
8. Зав. склад не может видеть расходы и прибыль.
9. Менеджер не может видеть расходы, себестоимость и P&L.
10. Все попытки доступа к запрещенным API должны возвращать 403.

14. MVP-реализация ролей

Для первой версии реализовать именно эти роли:

```
SUPER_ADMIN  
BOSS  
WAREHOUSE_MANAGER  
SHIFT_MASTER  
SALES_MANAGER
```

Не добавлять лишние роли, чтобы не усложнять систему.

Если потом бизнес вырастет, можно добавить:

```
ACCOUNTANT  
PURCHASER  
WORKER  
PRODUCTION_MANAGER
```

Но на старте лучше оставить 4 рабочие роли + Super Admin.

15. Критерии готовности блока авторизации и ролей

Блок считается готовым, если:

1. Пользователь может войти по email и паролю.
2. Неавторизованный пользователь не может попасть внутрь CRM.
3. Super Admin видит все.
4. Boss видит все бизнес-данные.
5. Зав. склад не видит деньги, расходы, оплаты, P&L.
6. Мастер смены видит только свои сменные отчеты.
7. Менеджер видит клиентов, заказы, отгрузки, оплаты и долги, но не видит расходы и прибыль.
8. Меню на frontend меняется под роль.
9. Backend возвращает 403 при попытке доступа без прав.
10. Пользователя можно отключить.
11. Boss не может создать или назначить Super Admin.
12. Все важные действия пишутся в audit_logs.